SafeLens Montgomery

Community Safety Intelligence Platform

Architecture & Build Plan

World Wide Vibes Hackathon | Public Safety, Emergency Response & City Analytics Track

City of Montgomery, Alabama | March 5-9, 2026

Bright Data	Gemini (Google AI)	City of Montgomery

1. Project Vision

SafeLens Montgomery is a bidirectional community safety intelligence platform that creates a live, context-rich communication channel between Montgomery residents and city officials/dispatchers. It combines the City of Montgomery's official open datasets with real-time web intelligence gathered through Bright Data, analyzed by AI, to surface not just what is happening — but why, and what it means for residents and responders.

At the heart of SafeLens is a recognition that Montgomery's police department is significantly understaffed. This is not simply a resource problem — it is a force multiplication challenge. SafeLens uses AI to amplify what each officer and dispatcher can see, prioritize, and act on, so that a lean team can operate with the situational awareness of a much larger one. It also directly targets 911 system efficiency: by surfacing context before a call is dispatched and enabling residents to self-report non-emergency incidents, SafeLens reduces pressure on overloaded dispatch queues and improves response times for calls that truly need an officer on the ground.

MISSION	Replace raw incident numbers with responsible, context-rich safety narratives that empower residents to stay informed, enable city officials to communicate proactively, and amplify the capabilities of an understaffed police department through AI-driven situational intelligence.

1.1 The Core Problem

Today, Montgomery residents experience a disconnect: 911 data exists in spreadsheets, news coverage is scattered, and city announcements live in separate channels. There is no unified view that connects these streams, and no way for residents to contribute real-time ground-level observations back to officials.

Pain Point	Impact
Police department understaffed	Officers cannot be everywhere — incidents go undetected or under-prioritized without AI assistance
911 hold times & overload	Dispatch queues are strained; non-emergency calls compete with genuine emergencies
No AI force multiplication	No system exists to help officers anticipate threats, prioritize responses, or identify patterns in real time
Residents have no context	See incident stats but not what they mean for their neighborhood
Officials lack two-way comms	No channel for proactive resident notification or feedback collection
News & data are siloed	Media coverage never connected to actual incident data patterns
No responsible framing	Raw crime maps create fear without context — bias in presentation

1.2 The Solution: SafeLens

SafeLens is a two-sided platform. Residents get a live, neighborhood-level safety dashboard with AI-narrated context. Officials and dispatchers get an operations panel to broadcast alerts, respond to

resident-submitted incidents, and see real-time community sentiment. Bright Data powers the live web intelligence layer that ties local news and public web signals to the official data.

1.3 AI as a Force Multiplier for an Understaffed Department

Montgomery's police department faces a structural staffing gap. SafeLens directly addresses this by making AI the connective tissue between data streams that officers currently have no bandwidth to monitor manually. Three specific capabilities are designed with this in mind:

Capability	How It Helps an Understaffed Department
Predictive Hotspot Alerts	AI analyzes 911 call patterns, time-of-day trends, and historical data to flag emerging hotspots before they escalate — allowing dispatchers to pre-position units rather than react after the fact
AI-Assisted Dispatch Triage	Incoming resident reports and 911 data are auto-classified by urgency and type. Dispatchers see a prioritized queue instead of a raw list — reducing cognitive load and speeding up response decisions for a small team
Non-Emergency Deflection	Residents can self-report non-emergency incidents (potholes, broken lights, minor property issues) directly through SafeLens. This deflects low-priority calls from the 911 queue entirely, freeing dispatcher capacity for genuine emergencies
Pattern Recognition Briefings	Each morning, officials receive an AI-generated briefing summarizing overnight activity, emerging patterns, and recommended focus areas — intelligence that would normally require an analyst to produce manually
911 Hold Time Reduction	By giving residents a self-service channel for non-emergency reports and proactive alerts for known situations, SafeLens reduces the volume of status-check calls to 911 — directly improving hold times for callers with real emergencies

2. Data Sources & Real Data Strategy

REQUIREMENT	The hackathon requires real data. SafeLens uses three distinct real-data pipelines: official City of Montgomery open data, live web scraping via Bright Data, and resident-submitted incident reports with geolocation.

2.1 Official City of Montgomery Open Data

Base URL: opendata.montgomeryal.gov | Format: ArcGIS REST API + GeoJSON + CSV

Dataset	Contents	Access Method	Refresh Rate
911 Calls Dataset	All calls received by dispatch	ArcGIS REST API	Real-time (hourly)
Crime Mapping / Incidents	Crime incidents by neighborhood and type	ArcGIS GeoJSON	Daily refresh
Crime Statistics	Aggregated trend data by category & year	CSV download	Monthly
Neighborhood Boundaries	GeoJSON polygons for 40+ neighborhoods	GeoJSON	Static
City Budget & Resource Data	Police/fire resource allocation by district	CSV	Annual

2.2 Bright Data Live Intelligence Layer

Bright Data tools are used to enrich the official data with real-time web context. This is the key differentiator that transforms a static dashboard into a living safety intelligence feed.

Tool	What it Scrapes	Used For	Cadence
SERP API	Google News scraping for "Montgomery AL safety" keywords	News correlation engine	Every 30 min
Web Unlocker	Deep-fetch Montgomery Advertiser and AL.com articles	Narrative AI context	Hourly
Scraping Browser	JS-heavy city portal pages, event calendars	Event-to-incident correlation	Every 2 hrs
MCP Server	Autonomous AI agent web research for incident context	AI narrative generation	On-demand

BRIGHT DATA BONUS	Using all four Bright Data tools (SERP, Web Unlocker, Scraping Browser, MCP) directly targets the bonus points criteria mentioned by Rafael Levi (Bright Data DevRel judge).

2.3 Resident-Submitted Incident Reports

Residents can submit observations via the SafeLens app — a pothole, a broken streetlight, suspicious activity, a traffic hazard. Each submission is geolocated, timestamped, and categorized. Officials can acknowledge, escalate, or close submissions with a public response. This creates the two-way communication channel central to the product.

3. System Architecture

3.1 Architecture Overview

SafeLens is a Next.js monorepo with four distinct layers: Data Ingestion, AI Analysis, API, and Frontend. The full-stack engineer owns the backend pipeline and API. Vibe coders own frontend components, prompts, and UI polish.

SAFELENS MONTGOMERY — SYSTEM ARCHITECTURE

LAYER 1: DATA INGESTION

opendata.montgomeryal.gov (ArcGIS REST) | Bright Data SERP API | Bright Data Web Unlocker | Bright Data Scraping Browser | Resident Submissions (Supabase)

↓ *Data flows into scheduled Python ingestion workers* ↓

LAYER 2: AI ANALYSIS ENGINE (Gemini 2.0 Flash + Claude Sonnet)

Incident Clustering by Neighborhood | News-to-Incident Correlation | Safety Narrative Generator | Alert Classification | Community Sentiment Scoring

↓ *Processed data stored in Supabase PostgreSQL* ↓

LAYER 3: API (Next.js API Routes / tRPC)

/api/incidents | /api/neighborhoods | /api/alerts | /api/news | /api/submit | /api/narrative | /api/broadcast

↓ *Served to frontend* ↓

LAYER 4: FRONTEND (Next.js 14 + Tailwind + Mapbox GL)

Resident View: Map + Safety Feed + Report Submission | Official View: Ops Panel + Alert Broadcast + Incident Queue

3.2 Tech Stack

Technology	Purpose	Owner	Priority
Next.js 14 (App Router)	Frontend + API routes	Full-stack engineer	Core
Tailwind CSS + shadcn/ui	UI component system	Vibe coders (Cursor/Lovable)	Core
Mapbox GL JS	Interactive safety map	Full-stack engineer	Core
Supabase (PostgreSQL)	Database, auth, realtime	Full-stack engineer	Core
Bright Data SDK (Python)	SERP, Unlocker, Browser, MCP	Full-stack engineer	Core
Gemini 2.0 Flash	Fast classification & clustering	Vibe coders (prompts)	AI
Claude Sonnet 4 (Anthropic)	Safety narrative generation	Vibe coders (prompts)	AI
Python FastAPI workers	Data ingestion cron jobs	Full-stack engineer	Backend
Vercel	Frontend deployment	Any team member	Deploy
Railway / Render	Python workers deployment	Full-stack engineer	Deploy

4. Features & User Flows

MVP PRINCIPLE	Features are split into two phases. Phase 1 (MVP) must be fully working before any Phase 2 work begins. The demo must be able to run on Phase 1 alone. Phase 2 features are enhancements that add depth and impress judges if time allows.

4.1 Phase 1 — MVP (Must Ship by End of Day 2)

These are the eight non-negotiable features that define SafeLens. If these work flawlessly, the product delivers its core value and is competitive for the prize.

MVP Feature 1: Data Ingestion Pipeline

- Pull 911 calls and crime incidents from opendata.montgomeryal.gov via ArcGIS REST API
- Store normalized records in Supabase: type, neighborhood, coordinates, timestamp
- Runs on a scheduled cron worker — this is the foundation everything else depends on
- GATE: No other feature is started until real data is confirmed flowing into Supabase

MVP Feature 2: Incident List View (No Map Required)

- Card/table display of real incidents grouped by neighborhood
- Each card shows: incident type, neighborhood, date, source badge (City of Montgomery)
- Filterable by neighborhood and incident type
- Intentionally built without the map first — unblocks all other frontend work immediately
- Map view replaces/enhances this in Phase 2 once data pipeline is confirmed stable

MVP Feature 3: Resident Report Submission

- Floating "Report Something" button — always visible on mobile and desktop
- Form: category (Suspicious Activity, Infrastructure, Traffic Hazard, Other), description, optional location
- Submits to Supabase resident_reports table with timestamp and auto-generated status of "Submitted"
- Report immediately appears in the incident list with a distinct "Community Report" badge
- No login required for residents — friction-free submission is essential

MVP Feature 4: Verified by City Badge

- Every resident report displays a live status badge: Submitted (grey) → Verified by City (blue) → Resolved (green)
- When an official acknowledges a report, the badge updates in real time via Supabase Realtime
- If the official adds a public response note (e.g. "Officer dispatched to location"), it appears under the report
- This single feature closes the communication loop — residents know officials are aware and acting
- Status history is preserved so residents can track progress over time

MVP Feature 5: Official Alert Broadcast

- Officials compose an alert: title, message body, severity level (Info / Warning / Emergency)
- Alert is written to Supabase alerts table and pushed via Supabase Realtime
- Resident view shows a persistent banner at the top of the page — appears within 1-2 seconds, no refresh needed
- This is the most visually compelling moment in the demo — plan to show it live

MVP Feature 6: Official Ops Panel

- Secure login via Supabase Auth — officials and dispatchers only
- Incoming resident report queue sorted by submission time and category
- One-click Acknowledge button → triggers the Verified badge on the resident side instantly
- Optional public response text field — what officials type here residents see under their report
- Resolve button closes the report and updates resident badge to Resolved (green)

MVP Feature 7: AI Safety Narrative

- When a resident clicks a neighborhood in the incident list, a side panel opens
- Claude Sonnet 4 generates a 2-3 sentence plain-language safety summary for that neighborhood
- Prompt is grounded in real incident counts, trend direction, and any available Bright Data news
- Responsible framing built into prompt — factual, context-aware, no alarmist language
- Cached in Supabase narratives table for 2 hours to avoid redundant API calls

MVP Feature 8: Bright Data SERP News Feed

- SERP API queries Google News for "Montgomery Alabama safety [neighborhood]" every 30 minutes
- Top 3-5 results per query stored in Supabase: headline, source, URL, published date
- News cards displayed in the resident view alongside incident data — each card links to the original article
- This satisfies the real-data and Bright Data integration requirements for the hackathon judges

4.2 Phase 2 — Enhancements (Day 3-4, if MVP is solid)

These features add significant depth and polish but are not required for the core value proposition. Build these only once every Phase 1 feature is working end-to-end.

Enhancement A: Interactive Map View

- Mapbox GL JS map replacing the incident list as the primary interface
- Heatmap layer for 911 call density; resident reports as distinct pins
- Click a neighborhood polygon to open the AI narrative side panel
- Color-coded zones: green / yellow / red based on trend vs. city average
- Implementation note: ~2 hours for fullstack engineer once data pipeline is stable

Enhancement B: Gemini Incident Classification

- Gemini 2.0 Flash classifies each incoming incident: Event-Driven, Seasonal, Ongoing, Single Incident
- Context tags appear on incident cards and in AI narratives
- Hotspot detection: flags neighborhoods with statistically elevated activity vs. 30-day rolling average

Enhancement C: Trend Analytics Panel

- 7/30/90-day incident trend charts (Recharts) for officials
- Correlation view: Bright Data news mention volume vs. 911 call volume
- AI-generated daily digest: morning briefing for dispatchers summarizing overnight activity

Enhancement D: Web Unlocker Article Deep-Fetch

- When a SERP result scores highly relevant, Web Unlocker fetches the full article text
- Full article context is passed to Claude Sonnet to enrich neighborhood narratives

Enhancement E: Scraping Browser — Event Calendar

- Scrapes Montgomery city event calendar (JS-rendered) to detect upcoming large events
- Auto-tags incident spikes near event dates as "Event-Driven" to prevent resident alarm

Enhancement F: MCP Deep Research Mode

- Officials can trigger an autonomous Bright Data MCP agent to research a specific incident pattern
- Agent browses the web and returns a structured research brief — no manual searching needed
- Stretch goal: only build if all other enhancements are complete

5. AI Strategy

KEY DIFFERENTIATOR	SafeLens uses a dual-model AI architecture: Gemini 2.0 Flash for fast, high-volume classification tasks (runs on every data ingestion cycle) and Claude Sonnet 4 for high-quality, responsible narrative generation (runs on demand).

5.1 Gemini 2.0 Flash — Ingestion-Time Classification

Role: Fast, cheap, runs every 30 minutes on all new data. Operated via Google AI Studio API.

Task	Description
Incident Clustering	Groups nearby incidents by type and proximity into meaningful clusters vs. isolated events
News-to-Incident Matching	Matches Bright Data SERP news articles to open data incident clusters (confidence score)
Context Tag Assignment	Labels each neighborhood event as: Event-Driven, Seasonal, Ongoing, or Single Incident
Hotspot Detection	Flags statistically elevated incident rates vs. 30-day rolling average per neighborhood
Submission Triage	Classifies resident submissions by urgency and routes to the right official queue

5.2 Claude Sonnet 4 — Narrative Generation

Role: Responsible, high-quality prose generation for resident-facing content. Called on demand when a user opens a neighborhood scorecard or when officials request a daily digest.

Task	Description
Neighborhood Safety Narrative	2-3 sentence plain-language explanation of what is happening in the neighborhood, grounded in data and stripped of alarmist framing
Official Daily Digest	A one-page summary of the city safety picture for the past 24 hours, generated for dispatchers each morning
Incident Context Brief	When a resident clicks a specific incident cluster, Claude generates a brief contextual explanation citing available news and historical data
Responsible Framing Check	Claude is prompted to flag and rephrase any output that could be seen as stigmatizing a community or amplifying fear beyond the data

SAMPLE PROMPT PATTERN	"You are a responsible public safety communicator for the City of Montgomery. Given the following data: [incident counts, trend, news headlines], generate a 2-sentence neighborhood safety update that is factual, context-aware, and avoids language that could stigmatize residents. Data: {context}"

6. Bright Data Integration Details

6.1 SERP API — News Correlation Engine

Query pattern: search Google News for "Montgomery Alabama [safety topic] [neighborhood name]" every 30 minutes. Parse top 5 results per query. Store headline, source, URL, and publication date in Supabase. Feed to Gemini for news-to-incident matching.

EXAMPLE QUERIES	"Montgomery Alabama crime Oak Park 2026" | "Montgomery AL 911 calls this week" | "Montgomery police incident [date]" | "Montgomery Alabama community safety news"

6.2 Web Unlocker — Article Deep-Fetch

When Gemini scores a news article as highly relevant to an open data cluster (confidence > 0.75), Web Unlocker fetches the full article text. This text is passed to Claude Sonnet as context when generating neighborhood narratives — grounding the AI output in real reporting.

6.3 Scraping Browser — City Event Calendar

Montgomery's events calendar and city council announcements are JavaScript-rendered pages. The Scraping Browser (Playwright-based) renders these pages and extracts upcoming events. Large events (concerts, festivals, games) are correlated to expected 911 call volume increases, allowing the system to auto-tag those as "Event-Driven" rather than alarming residents unnecessarily.

6.4 MCP Server — AI Agent Research Mode

An optional "Deep Research" feature for officials: an MCP-powered AI agent that can autonomously browse the web to research a specific incident or neighborhood pattern. The agent uses Bright Data MCP tools (search_engine, scrape_as_markdown) to gather context and returns a structured research brief. This is an on-demand, high-value capability for dispatcher briefings.

7. Database Schema (Supabase PostgreSQL)

Table	Key Columns	Purpose	Type
incidents	id, source, type, lat, lng, neighborhood_id, occurred_at, raw_data	Official open data 911 & crime records	Core
neighborhoods	id, name, geojson, avg_incident_rate, safety_score	Montgomery neighborhood polygons	Core
news_articles	id, headline, source, url, published_at, relevance_score, matched_incident_ids	Bright Data SERP results	Core
resident_reports	id, category, description, lat, lng, status (submitted/verified/resolved), verified_by, verified_at, official_response, created_at	Community submissions + verification loop	Core
alerts	id, title, body, severity, neighborhoods, created_by, created_at, is_active	Official broadcasts	Core
narratives	id, neighborhood_id, content, generated_at, model_used, source_data_hash	AI-generated safety text	Core
users	id, role (resident/official/dispatcher), email, created_at	Auth via Supabase Auth	Auth

8. Day-by-Day Build Plan

TIMELINE	Hackathon runs Thursday March 5 (9am Central) through Monday March 9 (9am Central). Approximately 96 hours of build time. Rule: no Phase 2 work starts until all 8 MVP features are working end-to-end. A polished MVP beats a half-finished feature-rich app every time.

Day 1 — Thursday March 5: Data Foundation (MVP Features 1-2)

GATE	Nothing else starts until real incident data is confirmed flowing into Supabase. This is the foundation everything sits on.

- Full-stack engineer: Initialize Next.js 14 repo, Supabase project, environment variables
- Full-stack engineer: Build data ingestion worker — fetch 911 + crime data from opendata.montgomeryal.gov ArcGIS REST API
- Full-stack engineer: Normalize and store incidents in Supabase (type, neighborhood, lat/lng, timestamp)
- Full-stack engineer: Set up SERP API cron — first real Montgomery news data flowing (MVP Feature 8)
- Full-stack engineer: Build core API routes: /api/incidents, /api/neighborhoods, /api/news
- Vibe coders: Scaffold app layout, color system, typography in Cursor/Lovable
- Vibe coders: Build incident list/card view consuming /api/incidents (MVP Feature 2)
- Vibe coders: Build news feed panel consuming /api/news
- END OF DAY GATE: Real Montgomery incident data visible in the app. SERP news cards rendering. Do not proceed to Day 2 until this is confirmed.

Day 2 — Friday March 6: Communication Loop (MVP Features 3-6)

GOAL	By end of Day 2, the full resident-to-official communication loop must be working: resident submits → official sees it → official acknowledges → resident sees Verified badge. This is the heart of the product.

- Full-stack engineer: Supabase Auth setup — official/dispatcher login
- Full-stack engineer: Build /api/submit, /api/acknowledge, /api/resolve, /api/broadcast routes
- Full-stack engineer: Configure Supabase Realtime on resident_reports and alerts tables
- Vibe coders: Build resident report submission form (MVP Feature 3)
- Vibe coders: Build Verified by City badge component with live status updates (MVP Feature 4)
- Vibe coders: Build official login page + ops panel report queue (MVP Feature 6)
- Vibe coders: Build acknowledge/resolve buttons wired to API — badge updates live on resident side
- Vibe coders: Build alert broadcast form for officials (MVP Feature 5)
- Vibe coders: Build alert banner on resident view with Supabase Realtime subscription
- END OF DAY GATE: Full loop demonstrated — resident submits report, official acknowledges, badge updates live, official broadcasts alert, resident sees it instantly.

Day 3 — Saturday March 7: AI + Polish (MVP Features 7-8 + Phase 2 Start)

GOAL	Complete the MVP with AI narratives, then begin Phase 2 enhancements. By end of Day 3 the app should be demo-ready as a complete MVP.

- Full-stack engineer: Wire Claude Sonnet 4 API call for neighborhood narrative generation (MVP Feature 7)
- Full-stack engineer: Cache narratives in Supabase to avoid redundant API calls
- Vibe coders: Build neighborhood detail side panel with AI narrative display
- Vibe coders: Mobile responsiveness pass — resident view must work on phone
- Vibe coders: Loading states, empty states, error handling throughout
- IF MVP IS SOLID — Phase 2: Full-stack engineer begins Mapbox map integration (Enhancement A)
- IF MVP IS SOLID — Phase 2: Vibe coders begin Gemini classification tags on incident cards (Enhancement B)
- END OF DAY GATE: Complete MVP working end-to-end. App is demo-ready. Everything from Day 4 is bonus.

Day 4 — Sunday March 8: Enhancements + Demo Prep

- Map view: finalize Mapbox integration, heatmap layer, neighborhood click → AI narrative
- Web Unlocker: fetch full article text for top SERP results, enrich AI narratives (Enhancement D)
- Scraping Browser: event calendar integration, auto-tag event-driven spikes (Enhancement E)
- MCP deep research mode for officials if time permits (Enhancement F — stretch goal)
- Responsible AI data literacy panel in resident view
- Full UX polish pass: consistent spacing, typography, transitions
- Record demo video walkthrough — plan the script around the 5 key moments listed in Section 11
- Write submission description and impact statement

Day 5 — Monday March 9: Final Submission

- Final QA pass with real live data — test every MVP feature
- Deploy: Vercel (frontend) + Railway (Python ingestion workers)
- Set up cron-job.org keep-alive ping to /api/health to prevent Supabase auto-pause
- Submit by 9am Central — do not wait until the last minute

9. Team Roles & Responsibilities

Role	Responsibilities	Primary Tool	Timeline
Full-Stack Engineer	Data pipeline, API routes, Supabase schema, Bright Data SDK integration, deployment	Backend + infrastructure	All 5 days
Vibe Coder — UI Lead	Component design in Cursor/Lovable, incident list view, report submission form, verified badge, alert banner, resident view layout and mobile responsiveness	Frontend (Cursor/Lovable)	Days 1-4
Vibe Coder — AI/Prompts	Claude & Gemini prompt engineering, narrative card component, AI response display	AI integration (AI Studio)	Days 2-4
Vibe Coder — Demo & Docs	Demo script, submission write-up, README, slide deck if needed, social posts	Non-code deliverables	Days 3-5

10. Responsible AI & Ethics Framework

The hackathon track explicitly calls for "responsible analytics." SafeLens addresses this head-on with four concrete design decisions.

Design Decision	How It Addresses Responsible Analytics
Context over raw counts	Incidents are always shown with trend direction and city average context — never as standalone scary numbers
No stigmatization language	Claude is explicitly prompted to avoid language that could stigmatize neighborhoods or their residents
Data source transparency	Every data point shows its source badge (City of Montgomery, Resident Report, News article)
Event-Driven auto-tagging	Spikes near known events are automatically tagged so residents are not alarmed by temporary increases
Data literacy panel	A permanent "How to Read This" panel explains what reported incidents do and do not represent
Bias acknowledgment	The platform explicitly notes that 911 call data reflects reporting patterns, not necessarily actual safety

11. Judging Criteria Alignment

Criterion	How SafeLens Addresses It	Strength
Relevance	Directly addresses the Public Safety track brief — 911 data + public trends + community context	Strong

Criterion	How SafeLens Addresses It	Strength
Execution Quality	Production-ready Next.js app, Supabase backend, real deployments on Vercel + Railway	Strong
Originality	Two-sided resident/official communication channel is novel vs. typical one-way dashboards	Strong
Social Impact	Serves both residents (safety awareness) and city officials (operational efficiency + community trust)	Strong
Commercial Potential	SaaS model: deployable to any mid-sized US city — Montgomery is the pilot	Strong
Bright Data Usage	All 4 tools used: SERP, Web Unlocker, Scraping Browser, MCP — maximizes sponsor bonus points	Strong

12. Deployment Plan

Component	Platform & Notes
Frontend (Next.js)	Vercel — one-click deploy from GitHub, free tier, custom domain optional
Database	Supabase — free tier with keep-alive ping via cron-job.org hitting /api/health every 4 days to prevent auto-pause. Includes Auth, Realtime, and Storage in one platform.
Python Workers	Railway.app — free trial, deploy Python FastAPI cron workers in minutes
Environment Variables	BRIGHTDATA_API_KEY, SUPABASE_URL, SUPABASE_ANON_KEY, GEMINI_API_KEY, ANTHROPIC_API_KEY
Demo URL	safelens-montgomery.vercel.app (or custom domain if time permits)

SafeLens Montgomery

Making Montgomery's communities safer through data, context, and connection.